ASSIGNMENT 3
SOFTWARE TESTING

Test Planning, Test Cases, Bug Reporting, Metrics and Design Techniques

Submitted By: Palak

Roll No.: 253028056

Class: MCA 3 Sem

Academic Assignment

1. What is Test Planning? Explain the main steps involved in the test planning process.

Test planning is the process of deciding in advance what will be tested, how testing will be performed, who will perform it, and when it will be completed. It helps ensure that testing is systematic, cost-effective, and aligned with project requirements.

Main steps:

1. Understand requirements: Study the software requirements, scope, and expected behavior.

2. Define test objectives and scope: Identify features to be tested and features that are excluded.

3. Select the test approach: Decide testing levels, types, tools, and techniques.

4. Estimate resources: Determine people, hardware, software, test data, and environment needs.

5. Prepare the schedule: Set milestones, task durations, dependencies, and deadlines.

6. Identify risks: List possible problems and plan ways to reduce their impact.

7. Define deliverables and criteria: Specify test cases, reports, entry criteria, and exit criteria.

8. Review and approve: Get feedback from stakeholders and finalize the plan.

A good test plan provides direction and helps track testing progress.

2. What is a Test Strategy? Explain its importance in software testing.

A test strategy is a high-level document that describes the general approach an organization or project will use for testing software. It defines the testing principles, methods, levels, tools, and quality goals.

Importance:

• It provides a consistent approach to testing across a project or organization.

• It helps teams select suitable testing techniques and tools.

• It clarifies the roles and responsibilities of testers and other team members.

• It supports planning of time, budget, and resources.

• It identifies major risks and explains how testing will address them.

• It helps stakeholders understand how software quality will be evaluated.

For example, a strategy may state that a web application will use unit, integration, system, security, and usability testing, with automation for repeated regression tests. The strategy gives overall direction, while detailed plans explain how the approach will be applied to a particular release.

3. What is a Test Plan? Explain the major contents of a Test Plan.

A test plan is a document that describes the scope, objectives, resources, schedule, and activities required to test a particular software product or release. It acts as a working guide for the testing team.

Major contents:

1. Test plan identifier and introduction: Name, version, purpose, and background.

2. Objectives and scope: What testing aims to achieve and which features are included or excluded.

3. Test items and features: Modules, interfaces, and functions to be tested.

4. Test approach: Test levels, types, techniques, and tools.

5. Test environment: Required hardware, software, browsers, devices, and configurations.

6. Roles and responsibilities: Duties of testers, developers, and managers.

7. Test data and deliverables: Data requirements, test cases, scripts, logs, and reports.

8. Schedule and estimates: Tasks, dates, milestones, and effort.

9. Risks and contingencies: Possible obstacles and backup actions.

10. Entry and exit criteria: Conditions for starting and completing testing.

These contents make testing organized, measurable, and easier to manage.

4. Differentiate between a Test Strategy and a Test Plan.

A test strategy and a test plan are related, but they serve different purposes.

Test Strategy:

• It is high-level and defines the overall testing approach.

• It focuses on principles, standards, test levels, techniques, and tools.

• It is often reusable across projects or maintained at organizational level.

• It is usually prepared by a test manager, QA lead, or quality team.

• It changes when the overall approach or major project conditions change.

Test Plan:

• It is more detailed and project- or release-specific.

• It explains the scope, tasks, resources, responsibilities, schedule, and deliverables.

• It is prepared for a particular product, project, or release.

• It is commonly created by a test manager or test lead with team input.

• It is updated as project scope, timelines, or resources change.

In short, the strategy explains the general way testing will be done, whereas the test plan explains the practical activities needed to carry it out.

5. What are the main goals and objectives of software testing? Explain.

The main goal of software testing is to evaluate software and find defects before they cause problems for users. Testing also provides information about quality and helps reduce the risk of failure.

Major objectives:

1. Detect defects: Find errors in requirements, design, code, and system behavior.

2. Verify requirements: Check whether the software meets specified requirements.

3. Validate user needs: Confirm that the product is useful and suitable for its intended purpose.

4. Improve quality: Assess reliability, performance, security, usability, and compatibility.

5. Prevent failures: Identify risks early so defects can be corrected at lower cost.

6. Build confidence: Provide evidence to stakeholders about the product's quality.

7. Support decisions: Supply test results that help decide whether a release is ready.

8. Check changes: Ensure fixes and new features do not break existing functions through regression testing.

Testing cannot prove that software is completely defect-free, but it can reduce uncertainty and the likelihood of failures.

6. What are Test Phases? Explain the different phases of software testing.

Test phases are stages through which testing activities are organized during the software development life cycle. A common sequence is:

1. Requirement analysis: Testers study requirements, identify testable conditions, clarify ambiguity, and assess risks.

2. Test planning: The team defines scope, strategy, resources, schedule, responsibilities, and deliverables.

3. Test case design and development: Test scenarios, test cases, scripts, and test data are prepared.

4. Test environment setup: Hardware, software, accounts, databases, and configurations needed for testing are prepared.

5. Test execution: Test cases are run, actual results are compared with expected results, and evidence is recorded.

6. Defect reporting and tracking: Differences are logged as defects, prioritized, assigned, fixed, and retested.

7. Retesting and regression testing: Fixed defects are checked again, and related features are tested for side effects.

8. Test closure: Results are summarized, completion criteria are checked, lessons learned are recorded, and reports are shared.

The exact phases and their order may vary according to the project and development model.

7. What factors should be considered while developing a Test Strategy?

A test strategy should be based on the project's purpose, risks, constraints, and quality expectations. Important factors include:

• Requirements and scope: The features, interfaces, and business rules that must be tested.

• Project complexity: Size, architecture, integrations, and technical difficulty.

• Risk and impact: The likelihood and consequences of failure, especially for critical functions.

• Quality attributes: Required security, performance, reliability, usability, and compatibility.

• Development model: Whether the project uses Agile, Waterfall, DevOps, or another approach.

• Testing levels and types: Unit, integration, system, acceptance, regression, and specialized testing.

• Tools and automation: Available frameworks, test management tools, and CI/CD systems.

• Resources and skills: Team size, experience, availability, and training needs.

• Environment and data: Test servers, devices, browsers, external services, and safe test data.

• Schedule and budget: Deadlines, cost limits, and release frequency.

• Standards and compliance: Legal, regulatory, privacy, and organizational requirements.

• Entry/exit criteria: Conditions for starting testing and deciding when it is sufficient.

Considering these factors helps create a realistic, risk-focused strategy.

8. Explain the resource requirements needed for software testing.

Software testing requires people, tools, environments, data, time, and budget. Planning these resources early prevents delays and improves test coverage.

1. Human resources: Test managers, manual testers, automation engineers, developers for defect fixing, domain experts, and security or performance specialists.

2. Hardware: Computers, servers, mobile devices, network equipment, and storage.

3. Software and tools: Test management systems, defect trackers, automation frameworks, browsers, API tools, performance tools, and version-control systems.

4. Test environment: A setup that resembles production, including operating systems, databases, services, configurations, and access permissions.

5. Test data: Valid, invalid, boundary, and realistic data. Sensitive information should be masked or replaced with safe test data.

6. Time and schedule: Sufficient time for planning, design, execution, defect fixing, retesting, and reporting.

7. Budget: Costs for staff, licenses, infrastructure, training, and external services.

8. Documentation: Requirements, test cases, test scripts, logs, and reports.

Resource requirements depend on the size, risk, and complexity of the software. They should be reviewed whenever the scope or schedule changes.

9. What is a Test Schedule? Explain the importance of scheduling in software testing.

A test schedule is a timeline that lists testing activities, their expected start and finish dates, duration, dependencies, and responsible team members. It may include milestones such as test design completion, test execution, regression testing, and final reporting.

Importance:

• It helps complete testing within the project deadline.

• It coordinates testing with development, builds, and release activities.

• It ensures that people and environments are available when needed.

• It makes dependencies and possible delays visible.

• It allows managers to track progress against planned milestones.

• It helps prioritize critical features when time is limited.

• It gives stakeholders a clear view of testing status and release readiness.

A useful schedule includes realistic estimates and some contingency for unexpected defects or environment issues. It should be updated when requirements, resources, or delivery dates change.

10. What is a Test Case? Explain the main components of a test case.

A test case is a documented set of conditions, inputs, actions, and expected results used to check whether a specific software function behaves correctly.

Main components:

1. Test case ID: A unique identifier, such as LOGIN_001.

2. Title or objective: A short description of what is being checked.

3. Requirement reference: The requirement or user story linked to the test.

4. Preconditions: Conditions that must be true before execution, such as an existing user account.

5. Test data: Inputs needed, such as username and password.

6. Test steps: Clear actions to perform in sequence.

7. Expected result: The correct behavior or output.

8. Actual result: What happened during execution.

9. Status: Pass, fail, blocked, or not run.

10. Priority and type: Importance and category, such as functional or regression.

11. Postconditions and notes: Any expected final state, evidence, or additional observations.

A well-written test case is clear, repeatable, traceable to requirements, and easy for another tester to execute.

11. Explain the process of designing and writing effective test cases.

Effective test cases are designed to cover important behavior clearly and efficiently.

Process:

1. Study requirements: Understand functional and non-functional requirements and clarify unclear points.

2. Identify test conditions: List features, workflows, rules, and possible failure situations.

3. Choose design techniques: Use equivalence partitioning, boundary value analysis, decision tables, state transitions, or experience-based testing as suitable.

4. Define preconditions and data: Specify the starting state and exact inputs.

5. Write steps: Use simple, numbered actions that can be followed consistently.

6. Specify expected results: Describe observable outcomes, including messages, data changes, or system responses.

7. Cover positive and negative cases: Test valid use, invalid input, missing fields, permissions, and error handling.

8. Prioritize: Give greater attention to high-risk and frequently used functions.

9. Review and trace: Check clarity, remove unnecessary duplication, and link each case to requirements.

10. Execute and maintain: Record results and update cases when the software or requirements change.

Effective test cases should be independent where possible, repeatable, unambiguous, and focused on one main objective.

12. What is Bug Reporting? Explain the important information included in a bug report.

Bug reporting is the process of documenting unexpected software behavior so the development team can reproduce, investigate, prioritize, and fix it. A clear report reduces misunderstanding and speeds up defect resolution.

Important information:

• Bug ID: A unique tracking number.

• Title: A concise summary of the issue.

• Environment: Build/version, operating system, browser/device, and relevant configuration.

• Preconditions: The state or setup required before reproducing the issue.

• Steps to reproduce: Exact, numbered actions that cause the bug.

• Expected result: What should happen according to requirements.

• Actual result: What happened instead.

• Severity: The impact of the defect on the system or user.

• Priority: How urgently it should be fixed.

• Attachments: Screenshots, videos, logs, request/response data, or error traces.

• Reporter and date: Who reported it and when.

• Status and assignee: Current workflow state and responsible person.

A good bug report is factual, specific, reproducible, and avoids vague descriptions such as “it does not work.”

13. Explain the Bug Life Cycle in software testing.

The bug life cycle describes the stages a defect passes through from discovery to closure. The exact status names may vary by organization.

Common stages:

1. New: A tester reports a defect for the first time.

2. Assigned: The defect is assigned to a developer or responsible team.

3. Open: The team reviews and accepts it for investigation.

4. In Progress: The developer investigates and works on a fix.

5. Fixed: A code change is made and a new build is provided.

6. Retest: The tester checks whether the reported problem has been corrected.

7. Verified: The tester confirms that the fix works as expected.

8. Closed: The defect is considered resolved and the record is closed.

Other possible statuses include Reopened (the issue still exists), Rejected/Invalid (not a genuine defect), Duplicate (already reported), Deferred (postponed), and Cannot Reproduce (the team cannot reproduce it with the available information).

A well-managed bug life cycle improves communication, accountability, and visibility into product quality.

14. What are Test Metrics? Explain their importance in measuring testing activities.

Test metrics are measurable values used to monitor testing progress, effectiveness, coverage, and product quality. They turn testing observations into information that supports project decisions.

Importance:

• Progress tracking: Shows how many test cases are planned, executed, passed, or blocked.

• Quality assessment: Helps reveal defect trends and areas with frequent failures.

• Coverage measurement: Indicates how much of the requirements or code has been tested.

• Productivity monitoring: Helps assess effort and testing throughput.

• Risk identification: Highlights modules with high defect counts or low coverage.

• Planning and forecasting: Supports estimates for remaining effort and release dates.

• Process improvement: Helps teams identify bottlenecks and improve future testing.

Metrics should be interpreted in context. For example, a high number of test cases executed does not automatically mean testing is effective if important risks remain uncovered. Metrics are most useful when combined with qualitative analysis and clear project goals.

15. Explain different types of testing metrics used in software testing.

Testing metrics can be grouped according to what they measure.

1. Test progress metrics: Number of test cases designed, executed, passed, failed, blocked, and remaining.

2. Test coverage metrics: Requirements coverage, feature coverage, code coverage, and risk coverage.

3. Defect metrics: Defect count, defect density, severity distribution, defect rejection rate, and defect leakage.

4. Test effectiveness metrics: Defect detection percentage, defects found by testing, and defects discovered after release.

5. Productivity metrics: Test cases executed per day, effort spent, and testing cost.

6. Reliability metrics: Failure frequency, mean time between failures (MTBF), and mean time to repair (MTTR), where relevant.

7. Automation metrics: Percentage of suitable tests automated, automation pass rate, and maintenance effort.

8. Schedule metrics: Planned versus actual completion dates and effort variance.

For example, requirement coverage can be calculated as (requirements covered by tests ÷ total testable requirements) × 100. Metrics should encourage better quality, not simply higher numbers.

16. What are Test Case Design Techniques? Explain their importance in software testing.

Test case design techniques are systematic methods used to select test inputs and scenarios. They help testers find defects without trying every possible combination.

Common techniques:

• Equivalence Partitioning: Divides inputs into groups expected to behave similarly; one or more representative values are tested from each group.

• Boundary Value Analysis: Tests values at and near the limits of an allowed range.

• Decision Table Testing: Tests combinations of conditions and their resulting actions.

• State Transition Testing: Checks behavior when the system moves between states.

• Use Case Testing: Derives tests from user workflows and interactions.

• Error Guessing: Uses experience to anticipate likely mistakes.

• Pairwise Testing: Selects combinations so that every pair of parameter values is covered.

Importance:

These techniques improve test coverage, reduce redundant cases, find edge-case defects, and make test design more structured. The appropriate technique depends on the requirements, risk, and complexity of the feature. Often, several techniques are combined for stronger coverage.

17. What is Equivalence Partitioning? Explain with a suitable example.

Equivalence Partitioning is a black-box test design technique that divides possible input values into groups, or partitions, expected to be processed in the same way. Instead of testing every value, a representative value is selected from each group.

Example: Suppose an application accepts an age from 18 to 60, inclusive.

• Invalid partition 1: Ages below 18 (for example, 15).

• Valid partition: Ages from 18 through 60 (for example, 35).

• Invalid partition 2: Ages above 60 (for example, 65).

A basic test set would use 15, 35, and 65 to represent the three partitions. Additional checks may be needed for blank input, letters, decimals, or other formats if the requirement permits them.

The technique reduces the number of test cases while maintaining useful coverage. It works especially well when inputs fall into clearly defined groups with similar expected behavior.

18. What is Boundary Value Analysis? Explain with a suitable example.

Boundary Value Analysis (BVA) is a black-box testing technique that focuses on values at the edges of an allowed range. Defects often occur near boundaries because of incorrect comparison operators, off-by-one errors, or mistaken limits.

Example: Suppose a form accepts an integer age from 18 to 60, inclusive. Important boundary tests are:

• 17: Just below the minimum; expected invalid.

• 18: Minimum allowed value; expected valid.

• 19: Just above the minimum; expected valid.

• 59: Just below the maximum; expected valid.

• 60: Maximum allowed value; expected valid.

• 61: Just above the maximum; expected invalid.

These values test both limits and the immediately neighboring values. BVA is useful for numeric ranges, lengths, dates, and quantities. It is often combined with equivalence partitioning to provide efficient and effective coverage.

19. Differentiate between Equivalence Partitioning and Boundary Value Analysis.

Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) are black-box techniques, but they focus on different aspects of input data.

Equivalence Partitioning:

• Divides the input domain into groups expected to behave similarly.

• Selects representative values from each group.

• Reduces the number of tests across valid and invalid categories.

• Example for age 18–60: test 15, 35, and 65.

Boundary Value Analysis:

• Focuses on the minimum and maximum limits and nearby values.

• Tests values just below, at, and just above boundaries.

• Is especially useful for ranges, lengths, and limits.

• Example for age 18–60: test 17, 18, 19, 59, 60, and 61.

The key difference is that EP tests representative categories, while BVA concentrates on edges where errors are common. Using both together usually provides better coverage than relying on only one technique.

20. What is Test Case Planning? Explain the steps involved in test case planning.

Test case planning is the activity of deciding which test cases are needed, how they will be organized, what resources they require, and when they will be prepared and executed. It connects project requirements with practical testing work.

Steps:

1. Review requirements: Understand the features, acceptance criteria, and constraints.

2. Identify test scenarios: List the workflows and conditions that need verification.

3. Set scope and priorities: Select cases based on risk, business importance, and available time.

4. Choose design techniques: Apply suitable methods such as equivalence partitioning, boundary value analysis, or decision tables.

5. Identify test data and preconditions: Decide what data and initial system states are needed.

6. Define expected results: Specify how success or failure will be recognized.

7. Assign responsibilities and estimates: Decide who will design, review, and execute cases and estimate effort.

8. Organize and trace cases: Give unique IDs and link tests to requirements.

9. Review and approve: Check coverage, clarity, feasibility, and duplication.

10. Schedule, execute, and maintain: Arrange execution, record results, and update cases as the product changes.

Good test case planning makes testing more complete, repeatable, and manageable.